

Student Attendance & Performance Tracker

An Android application for institution administration, faculty attendance and assessment management, and student progress monitoring.

Document control	Value
Identification	CUTM-SRS-SAPT-VER-0.1
Abbreviation	SAPT / Student Tracker
Document version	0.1 - academic review draft
Baseline date	1 October 2026
Application build version	1.0 (versionCode 1); production release pending
Backend constraint	Google Sheets + Google Apps Script
Prepared for	Seven-member project team and academic reviewers

Document purpose

Specify the system scope, functional behavior, interfaces, data structures, quality requirements and verification criteria. The section order follows the supplied Library Management System SRS sample; the content describes this project.

Implementation status legend

Built = present in the inspected source or documented setup. Planned = outstanding production requirement. Optional = candidate extension requiring a scope decision. Built does not imply live-device acceptance.

This specification includes ten labeled diagrams and flowcharts. They depict the baseline architecture unless explicitly labeled as a proposed extension.

Contents

Section	Page
1. Scope and identification	3
1.4 Environment; 1.8 development details	4-5
2. Referenced documents; 3.1 mode of operation	5
3.2 Technology stack and requirement inventory	6-8
3.2.3 Detailed functional requirements (27 IDs)	9-17
3.3-3.6 Configuration items and interfaces	18
3.7-3.9 Adaptation, safety and privacy	19
3.10 Non-functional requirements	20
4. Physical storage and logical data dictionary	21-22
5. Diagrams and flowcharts (10 figures)	23-32
6. Verification and acceptance	33
7. Roadmap, decisions and glossary	34

List of figures

1. System architecture - page 23
2. Use-case diagram - page 24
3. Data-flow diagram - context / level 0 - page 25
4. Data-flow diagram - level 1 - page 26
5. Logical entity relationship diagram - page 27
6. Login and authorization flowchart - page 28
7. Enrollment management flowchart - page 29
8. Attendance submission flowchart - page 30
9. Assessment draft/publication flowchart - page 31
10. Attendance request sequence diagram - page 32

1. Scope

The Student Attendance & Performance Tracker replaces fragmented manual attendance sheets and mark records with role-controlled Android workflows. Administrators maintain profiles, subjects and enrollment; faculty record attendance and publish assessments; students view their own progress.

1.1 Identification

Item	Specification
System title	Student Attendance & Performance Tracker
Document ID	CUTM-SRS-SAPT-VER-0.1
Package identifier	com.example.studenttracker
Document version / release	0.1 / review draft; not a production release approval
Actors	Administrator, faculty, student and backend operator

1.2 System overview

The frontend is written in Kotlin using Jetpack Compose and Material 3. Retrofit/OkHttp transports JSON POST requests to an Apps Script web app. The backend verifies Google identity, enforces permissions and stores academic records in a private Google spreadsheet.

The existing implementation uses Compose screen state, Kotlin coroutines and StateFlow, with a direct network layer. It does not currently implement a complete ViewModel/repository architecture, Room database, Hilt or WorkManager.

1.3 Document overview

Sections 1-4 follow the sample: scope, references, requirements and database backend. Sections 5-7 add compulsory system diagrams, acceptance/traceability and the production roadmap.

Scope boundary

Fees, library operations, biometric identity, admissions, payroll and general campus ERP functions are outside the baseline. Optional attendance/communication integrations are listed separately rather than assumed implemented.

1. Scope - environment and users

1.4 Software and hardware requirements

Environment	Requirement / recommendation
Development	Windows workstation, Android Studio, JDK 21 and Gradle wrapper; Node.js 22+ for backend bundling/tests.
Development hardware	Project recommendation: modern quad-core processor, 16 GB RAM and SSD with space for Android SDK/emulators. Not a measured minimum.
Client device	Android 7.0/API 24 or newer; Google account and compatible Google credential services. No camera requirement for baseline attendance.
Network	Internet required for authentication and current academic reads/writes. Offline queue is planned.
Cloud	Institution-controlled Google account, OAuth clients, Apps Script project and private spreadsheet.
Testing	Physical phone or emulator; real phone verification required for sign-in, sharing and presentation.

1.5 Brief software functional description

After Google sign-in and server role verification, the app opens an administrator, faculty or student workspace. Administrators prepare academic records and enroll students. Assigned faculty submit complete attendance rosters and assessment drafts, then publish complete results. Students receive only their own summaries and published marks.

1.6 Functional requirement groups

Data management: profiles, subjects, enrollment, attendance, assessments and audit records. Administration: dashboards, directories and safe deactivation. User work: assigned faculty workflows, student progress, reports, refresh and logout.

1.7 Configuration and operating boundary

Android receives the deployment base URL and Web OAuth client ID from external build configuration. Script Properties contain SPREADSHEET_ID and GOOGLE_WEB_CLIENT_ID, with optional ALLOWED_GOOGLE_DOMAIN. Institution-local dates use the Apps Script timezone. The current journal model is capacity-limited and must be load-tested before production use.

1.8 Development details / 2. References

1.8 Development details

Role	Responsibility
Seven-member development team	Implement, review, document and present the project; individual member names are not supplied.
Institution administrator	Approve accounts, maintain academic records and review reports.
Faculty users	Maintain attendance and assessments for assigned subjects.
Student users	View personal attendance and published performance.
Backend operator	Maintain cloud ownership, configuration, backups and releases.

2. Referenced documents

R1. Supplied SRS_Library_Management_System.pdf, 15 pages: structural and formatting example only. Library-specific requirements are not inherited.

R2. docs/FEATURE_CHECKLIST.md: implemented features, production work and optional backlog.

R3. PRODUCTION_PLAN.md and docs/STAGING_CHECKLIST.md: milestones and acceptance gates.

R4. docs/DEMO_SETUP_STATUS.md: deployment and demo-account evidence, with live verification limitations.

R5. Android source under app/src/main, app/build.gradle.kts and gradle/libs.versions.toml: client behavior and build configuration.

R6. backend/src/core.js, auth.js and apps-script.js: authorization, domain rules and physical storage.

R7. Backend and Android local test suites: recorded automated verification evidence.

3.1 Required mode of operation

Baseline operation is online. Google sign-in requires an active approved account; all academic operations pass through the authenticated backend. Tokens remain in memory and a fresh process requires sign-in. Configuration-change form retention is present; durable offline records and automatic synchronization are not.

3.2 System capability requirements

3.2.1 System software requirements

Category	Baseline technology / status
IDE and language	Android Studio; Kotlin
User interface	Jetpack Compose + Material 3; fixed dark theme
Asynchronous state	Kotlin coroutines; StateFlow for authentication; Compose state for screens
Navigation	Navigation Compose; stable encoded record IDs
Networking / serialization	Retrofit 2.11.0, OkHttp 4.12.0 and Gson
Authentication	Credential Manager + Google ID tokens; backend RSA signature verification
Local preferences	Preferences DataStore for metadata only; not an authenticated session store
Cloud storage	Google Sheets; TrackerUsers allowlist and TrackerRecords journal
Backend	Google Apps Script; JavaScript domain rules; bundled node-forge 1.4.0
Data exchange	Authenticated HTTPS JSON POST; API envelope version 2
Reports	CSV generation; narrow FileProvider sharing
Build settings	minSdk 24; compileSdk/targetSdk 37; JDK 21; Java compatibility 11
Testing / version control	JUnit, backend Node tests, Android lint; Git
Future reliability layer	Durable local database/queue and background synchronization: planned, technology not finalized

3.2.2 Application software requirements

The baseline shall support administrator profile/subject/enrollment management, assigned faculty attendance/assessment operations, and student-only progress visibility. It shall reject unauthorized or inconsistent writes and communicate validation, conflict and network failures clearly.

Document IDs below use CUTM-SRS-SAPT-XX. Status records implementation state; it is separate from priority and acceptance evidence.

3.2.2.1 Requirements table

ID suffix	Software requirement	Status
01	Application launch	Built
02	Google sign-in	Built
03	Role and profile authorization	Built
04	Session expiry and logout	Built
05	Administrator dashboard	Built
06	Student management	Built
07	Faculty management	Built
08	Subject management	Built
09	Safe deactivation	Built
10	Subject enrollment	Built
11	Faculty workspace	Built
12	Attendance capture	Built
13	Attendance persistence	Built
14	Attendance calculation	Built
15	Assessment drafts	Built
16	Assessment publication	Built
17	Student progress	Built
18	Reports and CSV sharing	Built
19	Loading, retry and form protection	Built
20	Audit journal and conflicts	Built
21	In-app account administration	Planned
22	Academic lifecycle and policies	Planned
23	Approved record corrections	Planned
24	Durable offline drafts and sync	Planned
25	Bulk import and advanced reporting	Planned
26	Operations and release readiness	Planned
27	Optional extensions	Optional

3.2.2.2 Brief description of requirements

Group / IDs	Required behavior
01-04: access	Launch; Google sign-in; server role/profile verification; expiry and logout.
05-09: administration	Real counts; student/faculty/subject directories; validated CRUD; history-preserving deactivation.
10-11: subject workflows	Administrator roster selection; faculty assignment scope; academic navigation.
12-14: attendance	Complete statuses; unique session submission; auditable snapshots; correct counted-session percentage.
15-16: assessments	Validated drafts; complete marks before publication; published record locking.
17-18: progress / reports	Own student summaries; authorized subject reports; safe CSV export.
19-20: interaction / integrity	Loading and retry feedback; unsaved-form guards; journal audits and conflict checks.
21-26: production work	Account administration, academic lifecycle, approved corrections, durable offline behavior, bulk import and operational release gates.
27: extensions	Separately approved optional enhancements; not baseline commitments.

Permissions matrix - baseline

Action	Admin	Faculty	Student
Manage profiles / subjects	Yes	No	No
Change enrollment	Yes	No	No
Attendance / assessments	Any active subject	Assigned only	No
Staff attendance report / CSV	Any active subject	Assigned only	No
Student progress endpoint	No	No	Own only
Spreadsheet operations	Authorized operator	No direct access	No direct access

Interface visibility is not a security boundary. Every API action must repeat the required role and record-scope check on the server.

3.2.3 Detailed functional requirements

CUTM-SRS-SAPT-01 | Application launch

Field	Specification
Status / actors	Built / All users
Input	Tap the installed app icon.
Process	Initialize Compose and configuration; use only a valid in-memory session for routing. A fresh process requires Google sign-in.
Output	Login screen or the verified role dashboard.
Acceptance	Saved email/role alone must never bypass login.

CUTM-SRS-SAPT-02 | Google sign-in

Field	Specification
Status / actors	Built / All users
Input	Select a Google account through Credential Manager.
Process	Obtain an ID token for the configured Web client; POST it to Apps Script; verify signature, issuer, audience, expiry and verified email.
Output	Authenticated identity or a readable authentication failure.
Acceptance	An altered, expired or wrong-audience token is rejected.

CUTM-SRS-SAPT-03 | Role and profile authorization

Field	Specification
Status / actors	Built / All users
Input	Verified Google claims and an allowlist entry.
Process	Require one active TrackerUsers mapping; apply role checks server-side. Faculty/student require active matching profiles. Scope faculty by assignment and students by their verified profile.
Output	Only authorized records/actions are returned.
Acceptance	An unknown account or forged student/subject identifier cannot expose restricted data.

3.2.3 Detailed functional requirements

CUTM-SRS-SAPT-04 | Session expiry and logout

Field	Specification
Status / actors	Built / All users
Input	Token expiry, unauthenticated response or Logout action.
Process	Clear the memory token and identity; clear saved session metadata on logout; rebuild navigation for the new authentication state.
Output	Return to login without previous-role access.
Acceptance	No token is persisted and a restarted app cannot authenticate from metadata.

CUTM-SRS-SAPT-05 | Administrator dashboard

Field	Specification
Status / actors	Built / Administrator
Input	Open the administrator home screen.
Process	Retrieve active student, faculty and subject counts; derive department count from active student department values; show management navigation.
Output	Current counts or explicit loading/error/empty feedback.
Acceptance	Counts match active records rather than hard-coded values.

CUTM-SRS-SAPT-06 | Student management

Field	Specification
Status / actors	Built / Administrator
Input	Student ID, name, email, department and optional academic/contact/guardian fields.
Process	Search, view, create and edit student records; validate required values and duplicate identifiers/emails; preserve stable IDs and check revisions.
Output	Saved profile, directory entry or actionable validation/conflict error.
Acceptance	Duplicate/archived IDs and stale edits are rejected; optional fields remain optional.

3.2.3 Detailed functional requirements

CUTM-SRS-SAPT-07 | Faculty management

Field	Specification
Status / actors	Built / Administrator
Input	Employee ID, name, email, department and optional employment/contact fields.
Process	Search, view, add and revise profiles; validate identity fields and check current revisions.
Output	Current faculty directory and saved profile.
Acceptance	A duplicate email/ID or stale revision cannot overwrite another record.

CUTM-SRS-SAPT-08 | Subject management

Field	Specification
Status / actors	Built / Administrator
Input	Subject code/name, active faculty ID, semester and planned class count.
Process	Validate faculty reference; require whole-number planned classes from 1 to 1000; save subject with immutable code and revision.
Output	Subject directory, details and assigned faculty link.
Acceptance	Invalid faculty references and out-of-range/non-integer class counts are rejected.

CUTM-SRS-SAPT-09 | Safe deactivation

Field	Specification
Status / actors	Built / Administrator
Input	Record ID and expected revision.
Process	Set active=false without removing historical journal entries; block faculty deactivation until active subjects are reassigned.
Output	Deactivated record or dependency/conflict message.
Acceptance	Past attendance/results remain stored after supported deactivation.

3.2.3 Detailed functional requirements

CUTM-SRS-SAPT-10 | Subject enrollment

Field	Specification
Status / actors	Built / Administrator
Input	Subject code, selected student IDs and current enrollment revision.
Process	Require distinct active students; allow at most 250; compare revision and persist the roster. Matching retries do not create a second change.
Output	Updated subject roster and revision.
Acceptance	Stale, duplicate or inactive roster entries are rejected.

CUTM-SRS-SAPT-11 | Faculty workspace

Field	Specification
Status / actors	Built / Faculty; administrator
Input	Open the academic hub.
Process	Load active subjects within the caller scope; offer attendance, assessment and report actions. Administrator also receives enrollment management.
Output	Authorized subject cards and actions.
Acceptance	Faculty cannot read/write a subject assigned to another faculty profile.

CUTM-SRS-SAPT-12 | Attendance capture

Field	Specification
Status / actors	Built / Faculty; administrator
Input	Authorized subject, date, slot and one status per enrolled student.
Process	Load minimal roster; mark present/absent/late/excused; support mark-all-present; validate real nonfuture date and complete current roster; confirm submission.
Output	Complete validated attendance payload or visible field/roster error.
Acceptance	Missing, duplicate, extra or invalid-status entries cannot be submitted.

3.2.3 Detailed functional requirements

CUTM-SRS-SAPT-13 | Attendance persistence

Field	Specification
Status / actors	Built / Faculty; administrator
Input	Attendance payload and stable request ID.
Process	Use subject/date/lowercase-slot as the unique key; append a session snapshot including student names. Accept an identical retry; reject a competing overwrite.
Output	Submission success, already-submitted success or conflict.
Acceptance	Retrying the same logical submission does not create duplicate attendance.

CUTM-SRS-SAPT-14 | Attendance calculation

Field	Specification
Status / actors	Built / Authorized report viewer; student
Input	Stored attendance session snapshots.
Process	Count present+late as attended and present+late+absent as counted; exclude excused. Use only sessions containing that student. Display N/A for zero denominator.
Output	Percentage rounded to one decimal with present and counted totals.
Acceptance	One present, one late, one absent and one excused produces 66.7%, not 50%.

CUTM-SRS-SAPT-15 | Assessment drafts

Field	Specification
Status / actors	Built / Faculty; administrator
Input	Assessment ID, subject, title, maximum marks, revision, request ID and complete roster marks.
Process	Validate maximum marks from 1 to 10000; accept scored 0..maximum, absent or pending; save unpublished draft; reject stale revision and mismatched request reuse.
Output	Saved draft with updated revision.
Acceptance	Zero is a scored value; pending and absent are distinct; out-of-range scores fail.

3.2.3 Detailed functional requirements

CUTM-SRS-SAPT-16 | Assessment publication

Field	Specification
Status / actors	Built / Faculty; administrator
Input	Completed draft and explicit publish confirmation.
Process	Require every entry scored or absent; save published=true; lock the published record against direct edits; protect student visibility.
Output	Published results accessible only to the relevant student and authorized staff.
Acceptance	Pending entries block publication; students cannot retrieve draft marks.

CUTM-SRS-SAPT-17 | Student progress

Field	Specification
Status / actors	Built / Student
Input	Open or refresh own dashboard.
Process	Derive student ID from the verified profile; retrieve own attendance summaries and own published mark entries.
Output	Personal attendance and results, including no-data states.
Acceptance	Supplying another student ID does not change the authenticated scope.

CUTM-SRS-SAPT-18 | Reports and CSV sharing

Field	Specification
Status / actors	Built / Administrator; assigned faculty
Input	Select an authorized subject and export its report.
Process	Calculate a consistent report snapshot; show current fixed 75% shortage rule; quote CSV fields and neutralize formula prefixes; share a temporary cache file with narrow read permission.
Output	Readable UTF-8 CSV and Android share chooser.
Acceptance	Unicode/commas/quotes remain valid; unauthorized report access fails.

3.2.3 Detailed functional requirements

CUTM-SRS-SAPT-19 | Loading, retry and form protection

Field	Specification
Status / actors	Built / All users
Input	Screen load, failed request, field edit or Back action.
Process	Display loading/error/empty states; retry on demand; disable busy writes; preserve supported configuration-change form state; warn before discarding dirty forms.
Output	Clear feedback and guarded navigation.
Acceptance	A failed request is not shown as an empty successful dataset; dirty Back asks before discarding.

CUTM-SRS-SAPT-20 | Audit journal and conflicts

Field	Specification
Status / actors	Built / Backend; authorized operators
Input	Authorized mutation and expected revision/request identity.
Process	Lock supported operations; append kind, ID, data, time, actor and action; reconstruct latest record; reject stale changes.
Output	Traceable journal update or conflict/busy response.
Acceptance	Concurrent stale edits cannot silently replace a newer version.

CUTM-SRS-SAPT-21 | In-app account administration

Field	Specification
Status / actors	Planned / Administrator
Input	Approved account email, role, profile mapping and lifecycle action.
Process	Provide account provisioning/suspension screens, integrity checks and audited role changes; preserve at least one recoverable administrator.
Output	Managed account status without routine manual spreadsheet editing.
Acceptance	Suspended users lose server access and incompatible mappings are refused.

3.2.3 Detailed functional requirements

CUTM-SRS-SAPT-22 | Academic lifecycle and policies

Field	Specification
Status / actors	Planned / Administrator
Input	Terms, programs, sections, enrollment dates and policy settings.
Process	Link subject offerings to academic periods; handle transfers, term closure and policy effective dates; calculate threshold/grade results using agreed rules.
Output	Term-aware records and configurable institutional policies.
Acceptance	A policy change does not silently rewrite closed historical results.

CUTM-SRS-SAPT-23 | Approved record corrections

Field	Specification
Status / actors	Planned / Student/requester; faculty; administrator
Input	Record reference, correction reason and proposed values.
Process	Track requests, verify approver permissions, preserve before/after versions and recalculate affected results after approval.
Output	Pending/approved/rejected request with traceable outcome.
Acceptance	Submitted attendance and published marks change only through the authorized correction workflow.

CUTM-SRS-SAPT-24 | Durable offline drafts and sync

Field	Specification
Status / actors	Planned / Faculty; administrator
Input	Draft or queued operation while offline; reconnection.
Process	Persist account-isolated drafts and stable operation IDs; revalidate permissions/rosters; retry temporary failures with bounded backoff; show conflicts for human resolution.
Output	Recovered drafts and explicit pending/synced/failed status.
Acceptance	Process death preserves drafts; sync does not duplicate writes or use last-write-wins for conflicts.

3.2.3 Detailed functional requirements

CUTM-SRS-SAPT-25 | Bulk import and advanced reporting

Field	Specification
Status / actors	Planned / Administrator; scoped report viewers
Input	Import file/template or report filters.
Process	Preview imports, validate rows and references, report errors; support term/department/section/date filters and printable branded progress PDFs.
Output	Approved imported rows, error report or filtered output.
Acceptance	Invalid rows cannot silently corrupt records; report scope matches caller permissions.

CUTM-SRS-SAPT-26 | Operations and release readiness

Field	Specification
Status / actors	Planned / Institution operator
Input	Backup schedule, staged release and support incident.
Process	Use separate environments; test restore, migrations and rollback; monitor failures/capacity; register release signing; document ownership/privacy/support.
Output	Recoverable, monitored deployment with accepted release evidence.
Acceptance	Restore and all-role device workflow pass before production sign-off.

CUTM-SRS-SAPT-27 | Optional extensions

Field	Specification
Status / actors	Optional / Institution-approved roles
Input	An explicitly approved extension request.
Process	Consider QR/location-assisted attendance, notifications, timetable, assignments, guardian access or analytics with scoped permissions and documented dependencies.
Output	Separately approved enhancement.
Acceptance	No optional feature is represented as implemented or required without an institution decision.

3.3-3.6 Configuration items and interfaces

3.3 Computer Software Configuration Items (CSCI)

Item	Responsibility
Android application	Compose screens, navigation, credential flow, API models, feedback and CSV sharing.
Backend domain/auth bundle	Token verification, role/scope rules, validation, calculations and retry/conflict semantics.
Apps Script adapter	HTTP dispatch, certificate cache, Script Properties, locking and spreadsheet access.
Spreadsheet configuration	Allowlist and append-only record journal; restricted direct operator access.
Build and verification assets	Gradle configuration, backend bundling, tests and deployment documentation.

3.4 External interface requirements

EI-01: Credential Manager obtains a Google ID token for the configured OAuth audience. EI-02: the app sends HTTPS JSON POST to the Apps Script deployment /exec endpoint. EI-03: Apps Script retrieves trusted Google signing certificates. EI-04: Apps Script accesses only its configured spreadsheet. EI-05: Android share targets receive temporary read access only to the selected exported file.

3.5 Internal interface requirements

Compose screens call suspend Retrofit APIs and interpret typed responses. AuthSession holds token and verified identity; DataStore holds metadata. TrackerInterceptor moves declared query fields into a JSON POST and adds the token. Apps Script authenticates the request, then createService applies domain permissions/rules using a sheetStore adapter.

3.6 Internal data requirements

Exchange fields shall use stable identifiers and documented JSON types. Missing optional profile fields may be blank; pending/absent marks use null score. Attendance snapshots retain student names. UTC journal timestamps and institution-local session dates serve different purposes and shall not be interchanged.

3.7-3.9 Adaptation, safety and privacy

3.7 Adaptation requirements

Baseline deployment parameters are spreadsheet ID, OAuth audience, deployment URL and script timezone, with an optional signed Google hosted-domain restriction. Academic threshold/grade policy, first-class terms and institution branding are planned adaptations. They are not currently configurable in-app.

3.8 Safety and academic data integrity

Incorrect attendance or marks can affect academic decisions. The system shall validate complete rosters, distinguish pending/absent/zero marks, reject stale updates and preserve historical evidence. Submitted attendance and published results shall not be silently overwritten. Production corrections require explicit approval and before/after audit records.

A network timeout must not be presented as proof that a write failed. The baseline has server retry deduplication; durable client recovery/status checking is still planned. Backup and restore verification are required before institution-wide reliance.

3.9 Security and privacy requirements

- SEC-01: Verify token signature and claims before reading academic data; reject unknown/inactive allowlist mappings.
- SEC-02: Enforce role, subject assignment and own-student scope on every backend action.
- SEC-03: Use HTTPS; keep tokens out of request URLs, exported files and device persistence.
- SEC-04: Validate request sizes and field bounds; return safe internal-error messages.
- SEC-05: Restrict direct spreadsheet sharing and cloud administrator privileges.
- SEC-06: Preserve actor/action/time audit data and define secure administrator recovery.
- SEC-07: Publish institution privacy/retention rules and minimize guardian/contact collection.
- SEC-08: Revalidate access for offline synchronization and clear prior-account cached data when that planned layer is added.

3.10 Non-functional requirements

The following are proposed measurable production acceptance targets. They are not measured guarantees of the current demo. Capacity and policy targets require institution agreement.

ID / quality	Target / verification
NFR-01 Security	All role/action and cross-user denial tests pass; no academic data returned to an unapproved principal.
NFR-02 Correctness	Agreed attendance/grade edge-case fixtures pass, including zero denominator, excused, zero marks and roster changes.
NFR-03 Integrity	Identical supported retries yield one logical write; stale concurrent updates return conflict rather than overwrite.
NFR-04 Response time	Proposed pilot target: p95 read ≤ 5 seconds and write ≤ 10 seconds on an agreed dataset/network; measure cold starts separately.
NFR-05 Capacity	Baseline guard: ≤ 250 enrolled students per subject and $\leq 40,000$ characters per saved aggregate. Measure smaller effective limits for real payloads; do not promise unlimited growth.
NFR-06 Recovery	Proposed pilot RPO ≤ 24 hours and RTO ≤ 4 hours; prove through a restore drill and institution-approved backup policy.
NFR-07 Usability	All three roles complete primary tasks without developer help; loading/failure/empty states remain distinguishable.
NFR-08 Accessibility	Verify TalkBack, 200% font scaling, non-color status meaning and appropriate touch targets on supported devices.
NFR-09 Compatibility	Test API 24 and a current supported Android device, plus representative phone/tablet and orientation configurations.
NFR-10 Maintainability	Reproducible client/backend builds; meaningful tests and lint in CI; documented API/schema changes and dependency ownership.
NFR-11 Availability	Monitor service failures and Google service limits; agree the supported operating hours and incident response rather than claiming an untested uptime SLA.
NFR-12 Privacy	No token/marks/guardian contact leakage in public logs; only authorized exports and institution-defined retention.

4. Database backend - physical storage

Google Sheets is the compulsory cloud data store. The current implementation creates exactly two application tabs. Logical academic entities are JSON records in a journal, not separate relational tables or one sheet per entity.

4.1 TrackerUsers - account allowlist

Column	Type / rule
email	Normalized Google account email; exactly one matching entry required.
role	admin, faculty or student.
profileId	Faculty employeeId or student rollNumber; blank permitted for admin.
name	Display name; Google claim may be used as fallback.
active	TRUE means authorized; other values are denied.
googleSubject	Optional pinned Google sub claim.

4.2 TrackerRecords - append-only journal

Column	Type / rule
kind	student, faculty, subject, enrollment, attendance or assessment.
id	JSON-encoded record key; attendance key is a JSON tuple.
data	JSON object containing the entity snapshot and relevant revision/state.
at	ISO timestamp of the journal write.
actor	JSON-encoded verified actor email for normal API mutations.
action	Domain action, such as saveAttendance or saveAssessment.

Latest state is reconstructed by scanning the journal in row order and retaining the last record for each (kind, id). Append-only here describes application behavior; a spreadsheet owner can still edit cells directly. Production controls must restrict and monitor that access.

4.3 Logical data dictionary and rules

Entity / identifier	Main stored fields and relationships
Student / rollNumber	name, email, phone, department, batch, year, semester, section, parentName, parentPhone, parentEmail; active, revision, updatedAt.
Faculty / employeeId	name, email, phone, department, designation, joiningDate; active, revision, updatedAt.
Subject / subjectCode	subjectName, facultyId -> Faculty, semester, totalClasses; active, revision, updatedAt.
Enrollment / subjectCode	studentIds[] -> Student; revision, updatedAt. One aggregate per subject, with zero or more current students.
Attendance / composite key	subjectCode -> Subject, date, slot, requestId, records[] {studentId, name, status}; updatedAt.
Assessment / assessmentId	subjectCode -> Subject, title, maxMarks, marks[] {studentId, name, status, score}, published, revision, requestId, updatedAt.

4.4 Keys and relationships

Subject.facultyId references an active Faculty at creation/update. Enrollment resolves active Student records. Attendance and assessment contain embedded roster/mark snapshots; their historical entries do not rely on current enrollment to retain evidence. Account.profileId is role-dependent and must match an active profile for faculty/student authorization.

4.5 Calculation and status rules

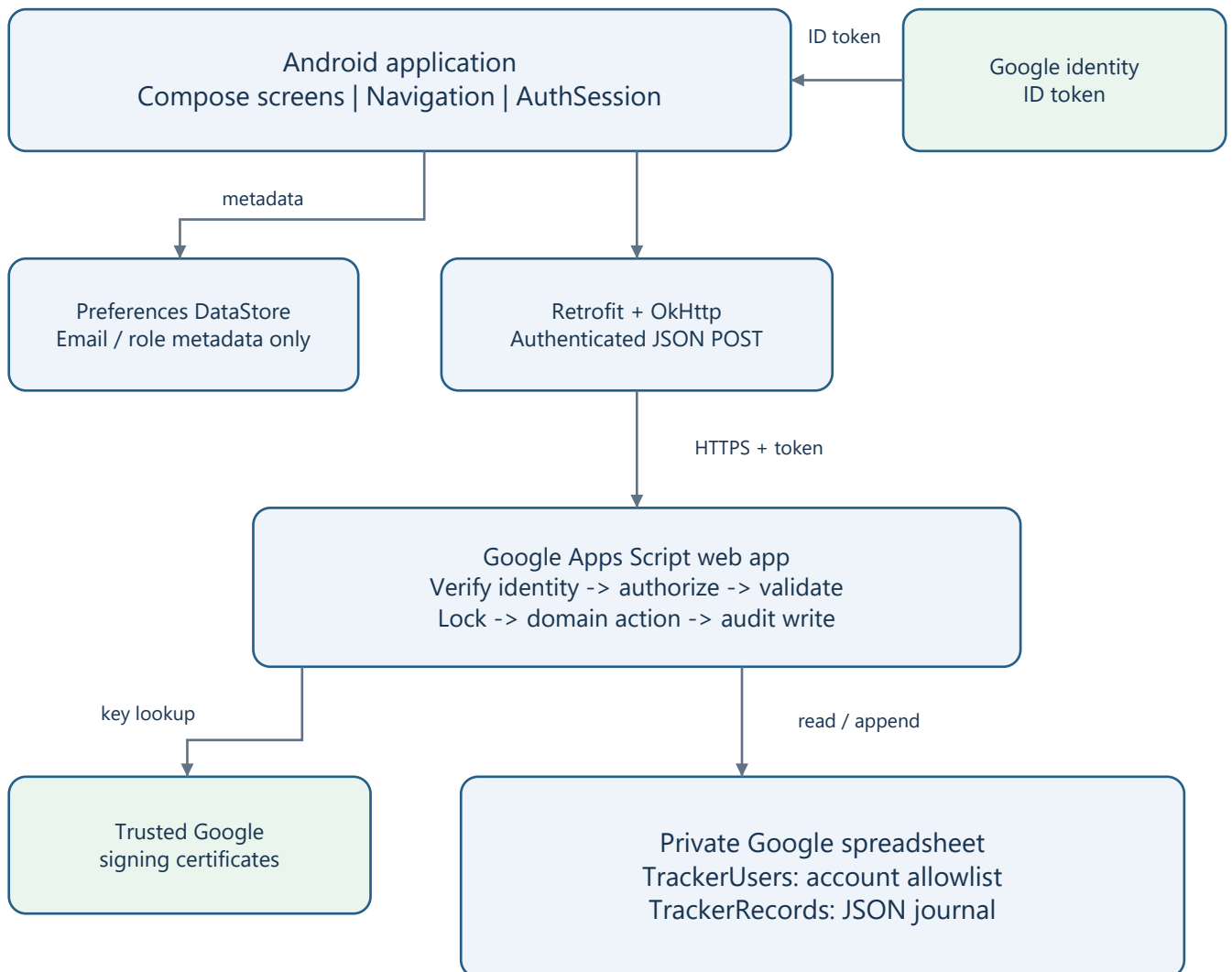
Attendance % = $100 \times (\text{present} + \text{late}) / (\text{present} + \text{late} + \text{absent})$. Excused entries are excluded; no denominator means N/A. Example: P=1, L=1, A=1, E=1 gives 66.7%. Planned subject.totalClasses is not the recorded attendance denominator.

Marks status = pending, absent or scored. Scored zero remains zero; pending/absent score is null. An assessment may be published only when no marks are pending. Weighted grades and retest policies are planned.

4.6 Planned schema evolution

Terms, subject offerings, dated enrollment, correction requests and durable sync metadata need versioned schema changes. Add migrations and projection recovery before changing production data; do not assume these entities already exist.

5.1 System architecture

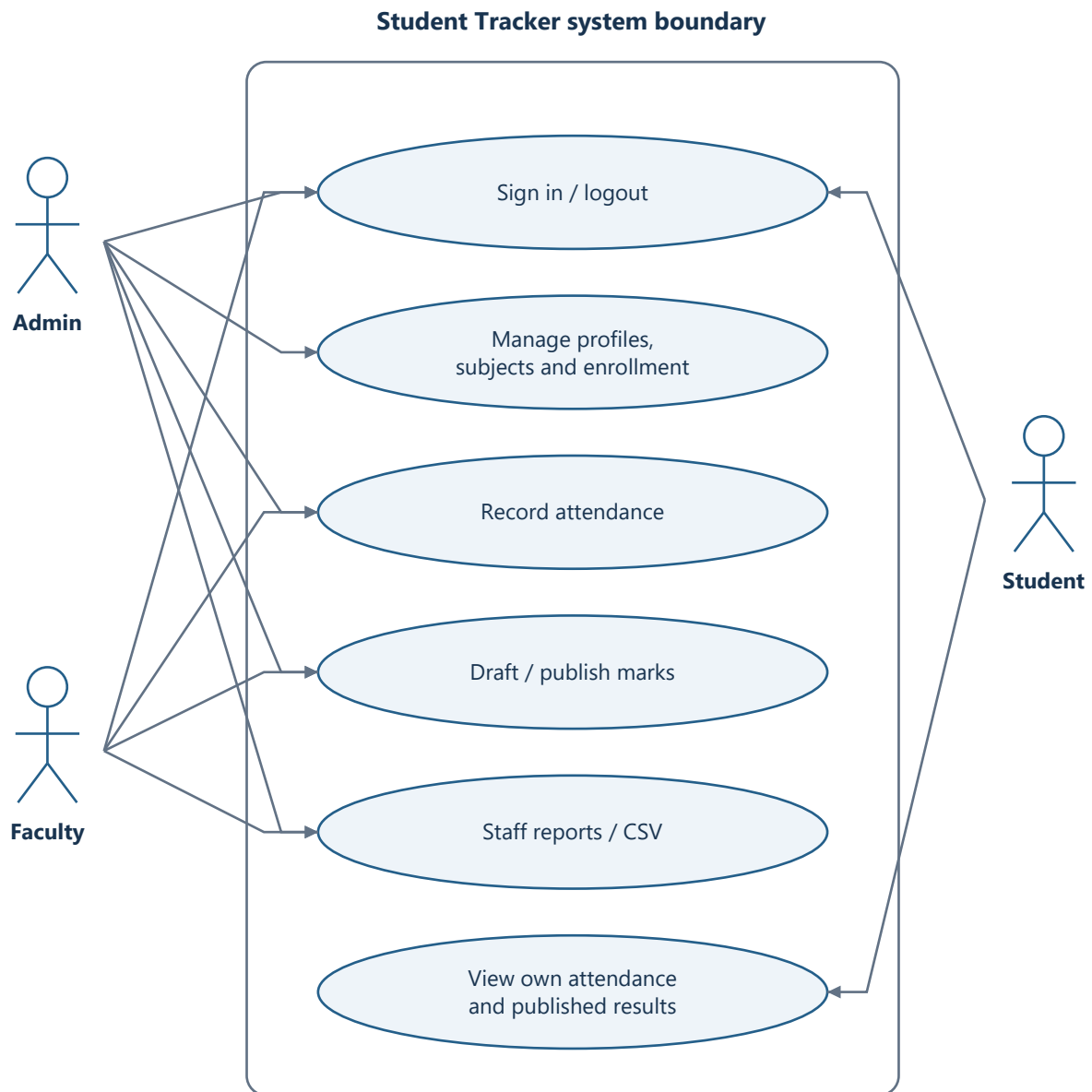


Trust boundary: device data is untrusted; backend checks every action.

Figure 1. System architecture

Android, Google identity, Apps Script and private Sheets are separated by explicit interfaces. Stored preferences are metadata, not an authenticated offline session.

5.2 Use-case diagram

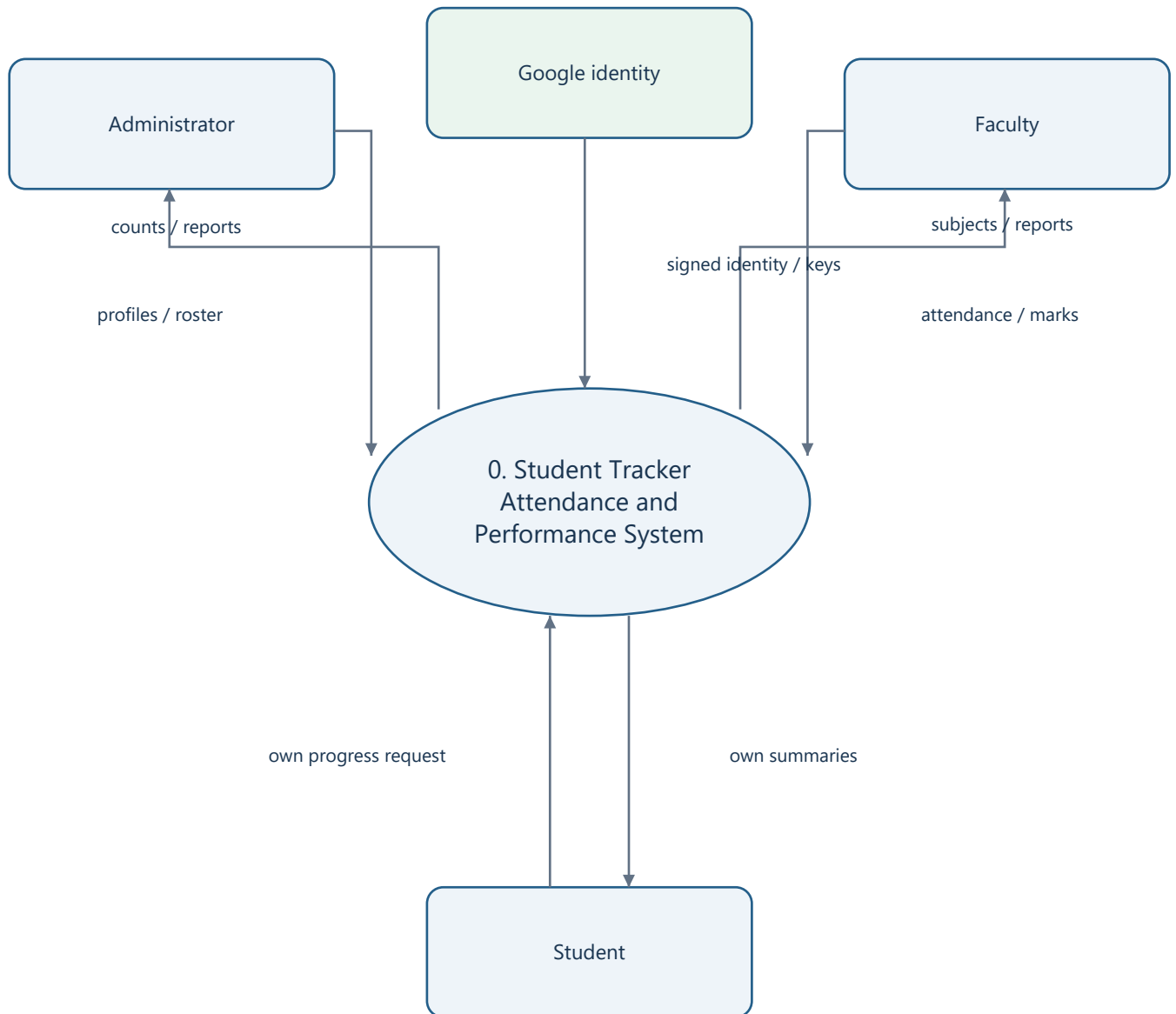


Staff actions are subject-scoped; student progress is own-record only.

Figure 2. Use-case diagram

Administrator and faculty staff actions overlap, but faculty actions are restricted to assigned subjects. Students view their own summaries and published results only.

5.3 Data-flow diagram - context / level 0

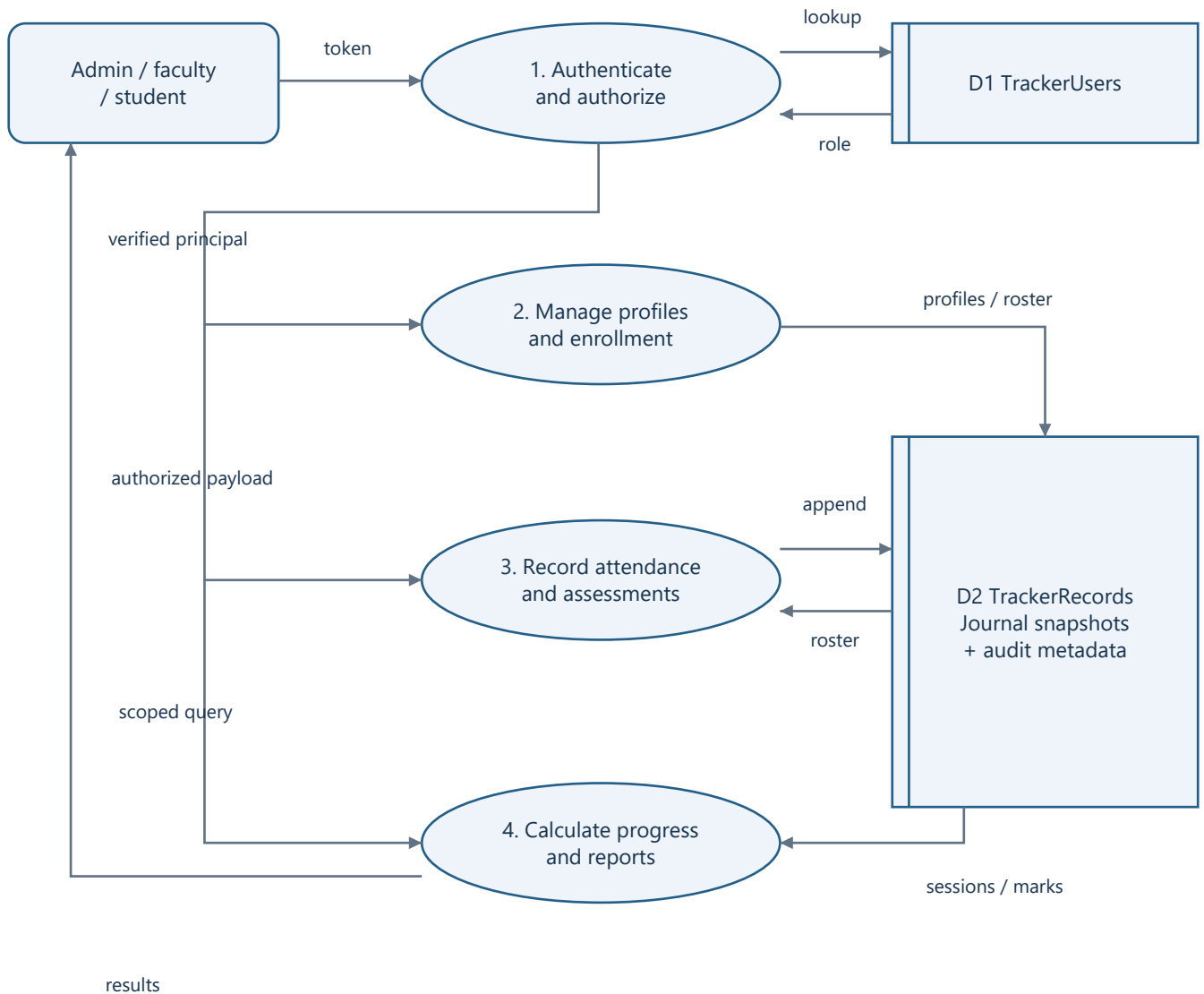


Context diagram: internal storage is intentionally inside process 0.

Figure 3. Data-flow diagram - context / level 0

Process 0 represents the complete Student Tracker system. Actors exchange management data, attendance/marks and own-progress requests; Google identity supplies trusted credentials.

5.4 Data-flow diagram - level 1

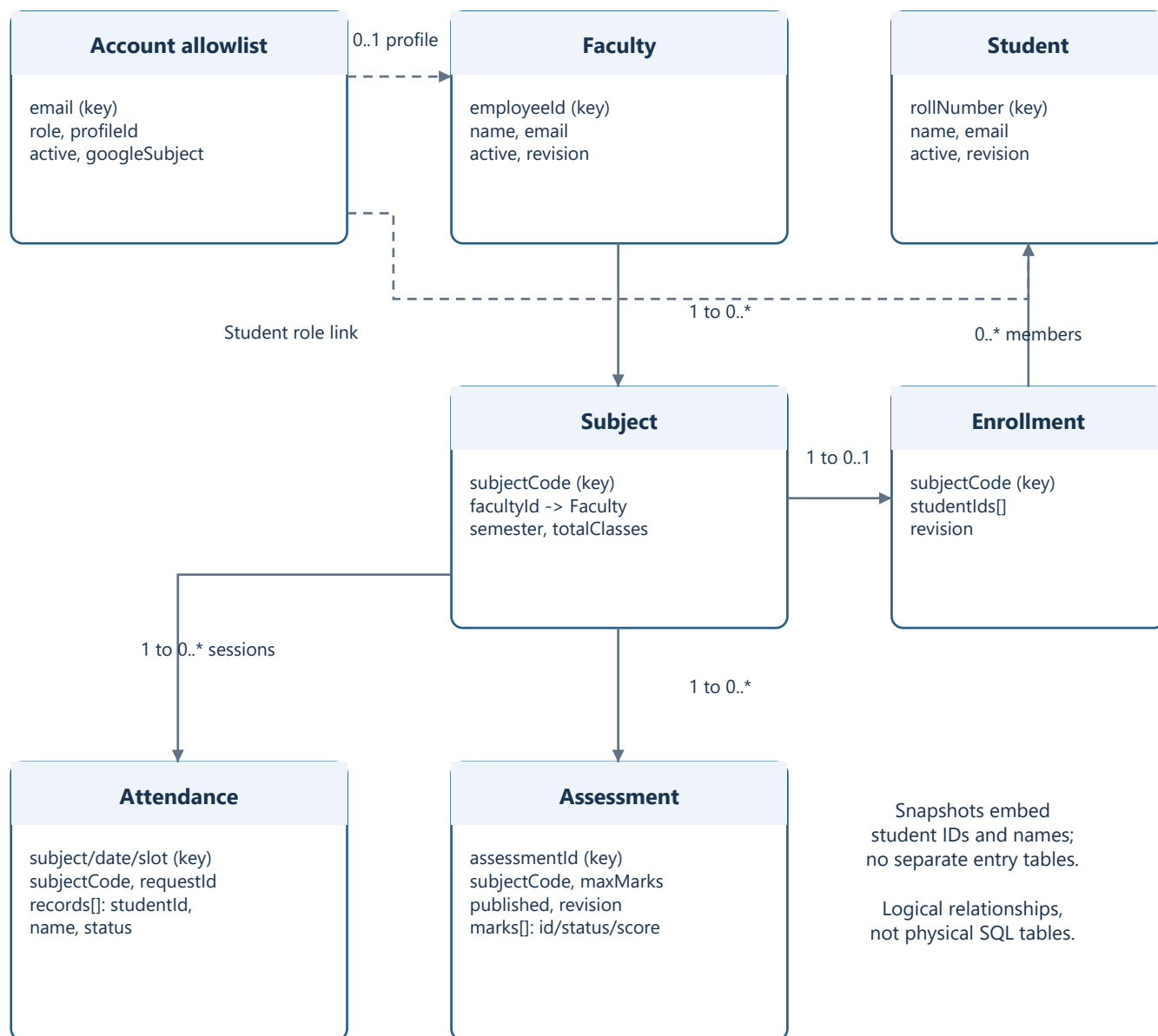


All mutation paths carry the verified actor and use the script lock.

Figure 4. Data-flow diagram - level 1

Authentication yields a verified principal. Profile/enrollment, attendance/assessment and report processes use the allowlist and journal data stores with server authorization.

5.5 Logical entity relationship diagram



All academic entity versions live in TrackerRecords JSON journal rows.

Figure 5. Logical entity relationship diagram

1 means one; 0..1 means optional single; 0..* means zero or more. Account profile links are conditional on role. Enrollment arrays and roster/mark entries are embedded JSON, not SQL tables. The physical backend still has only two tabs.

5.6 Login and authorization flowchart

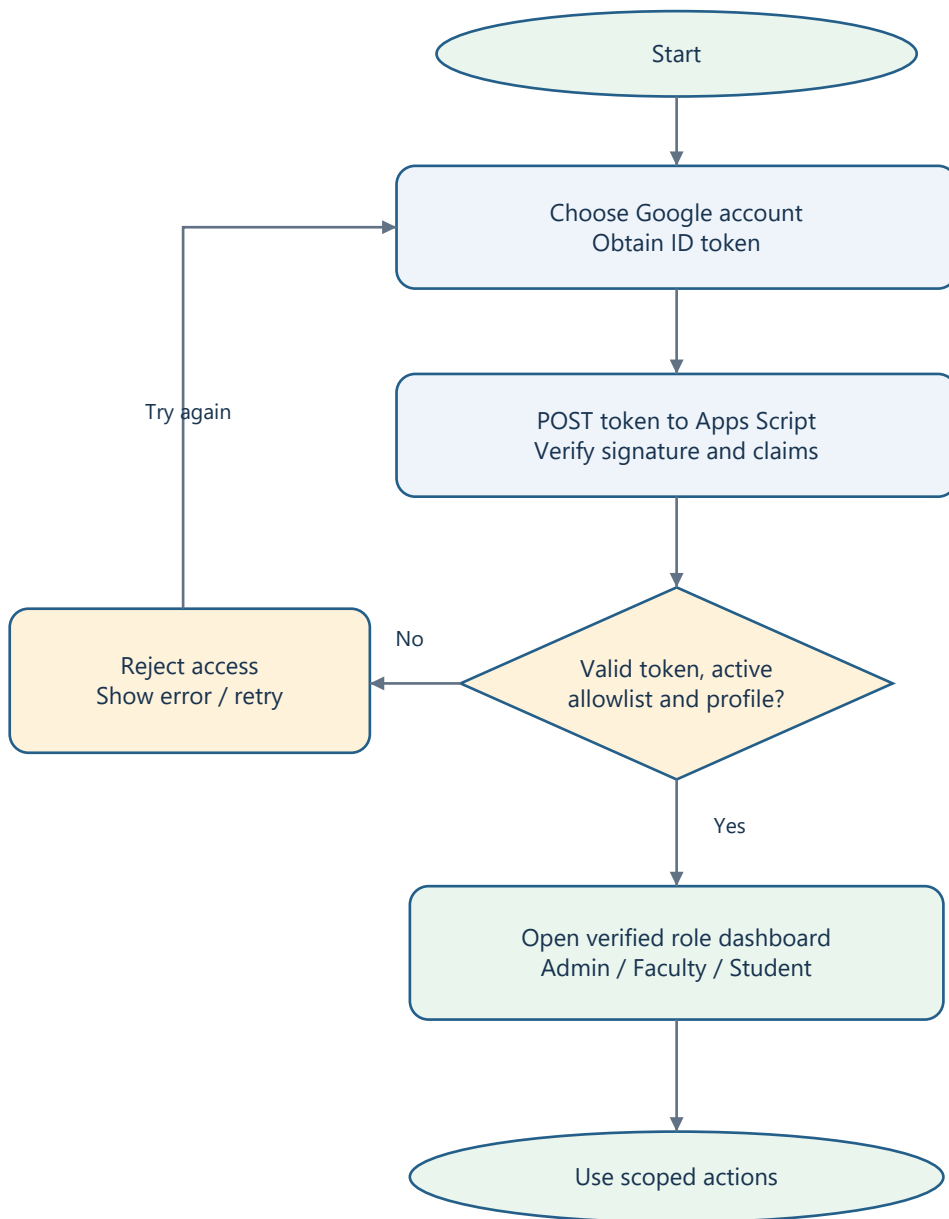


Figure 6. Login and authorization flowchart

No self-registration or role selection grants access. A valid Google identity must also match one active authorized mapping and, where required, an active profile.

5.7 Enrollment management flowchart

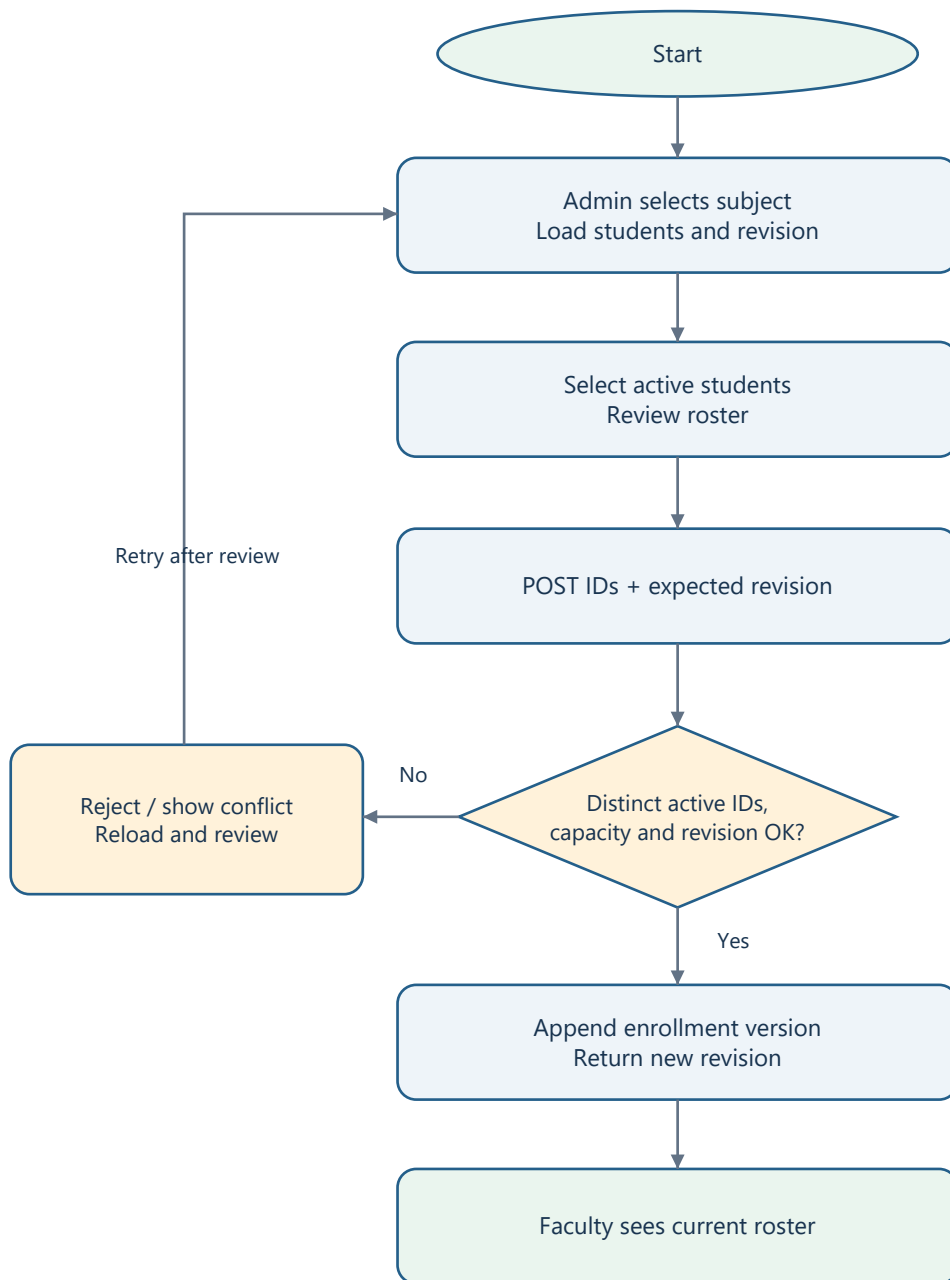


Figure 7. Enrollment management flowchart

Administrator edits carry the current revision. Stale edits require reloading and review; the server does not silently apply last-write-wins.

5.8 Attendance submission flowchart

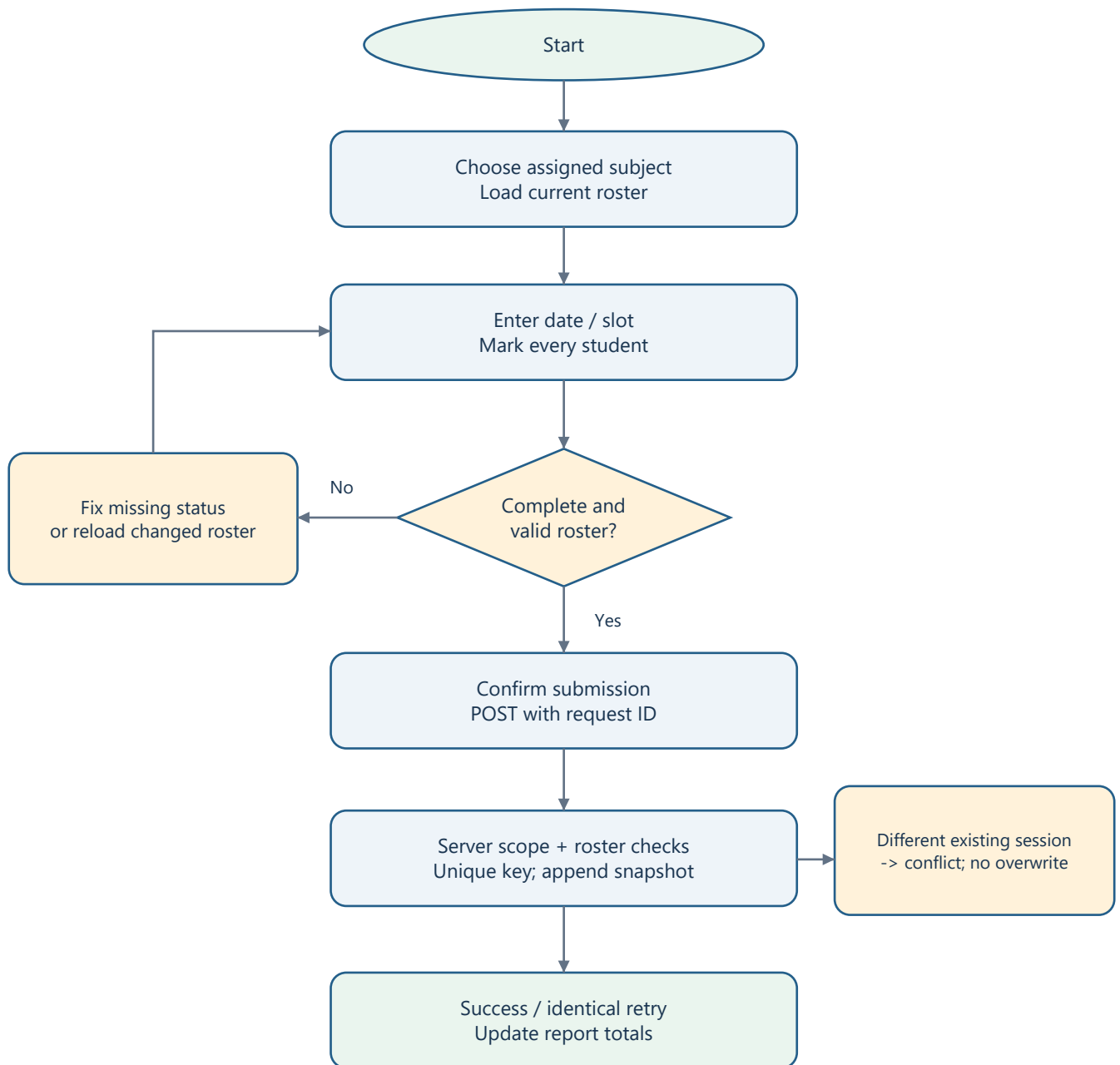


Figure 8. Attendance submission flowchart

A complete roster is confirmed before submission. The server deduplicates an identical retry and rejects another payload for an already submitted session.

5.9 Assessment draft/publication flowchart

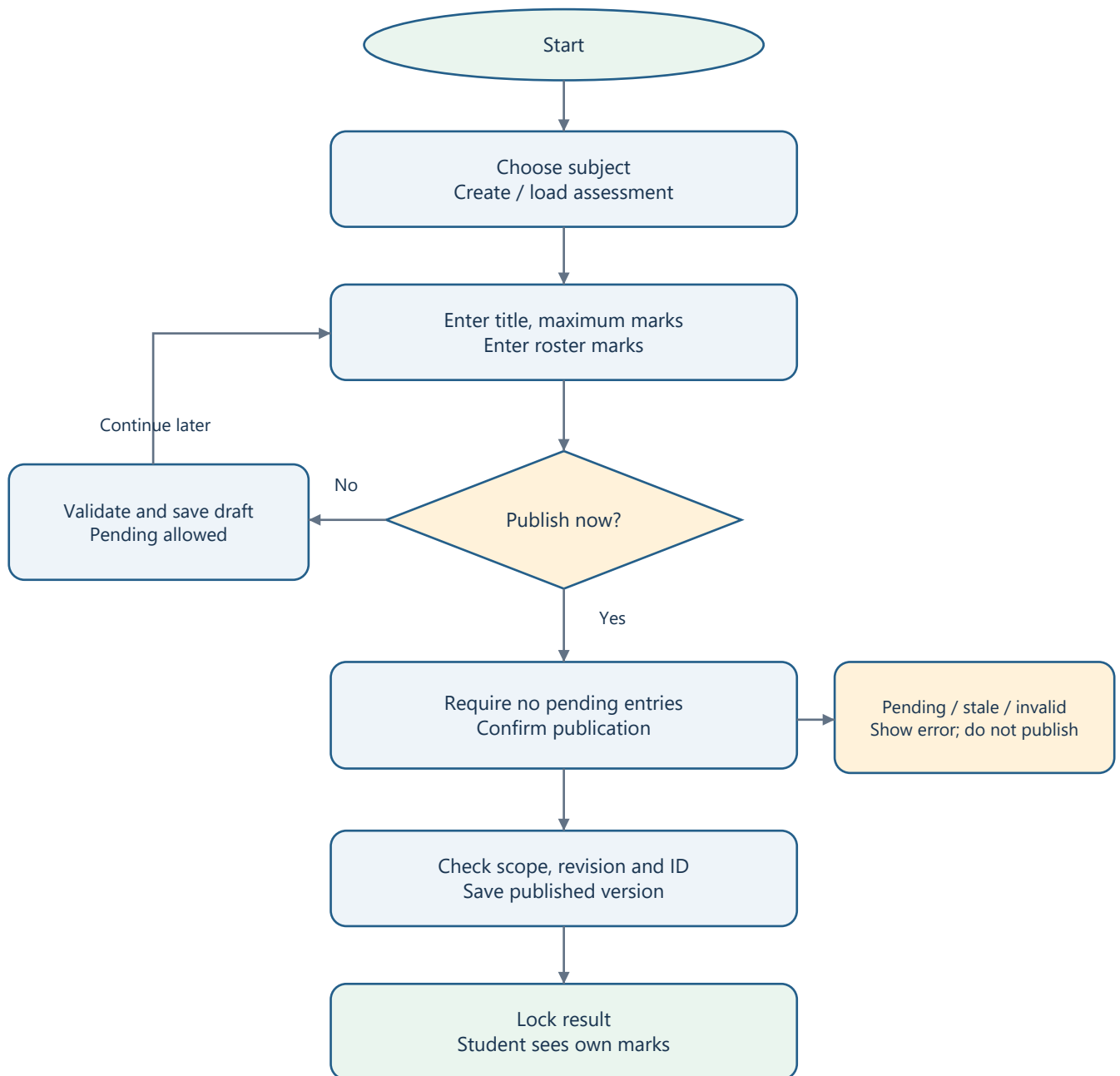


Figure 9. Assessment draft/publication flowchart

Pending marks are valid in drafts, but publication requires every mark scored or absent. Published assessments are locked; an approved correction workflow is planned.

5.10 Attendance request sequence diagram

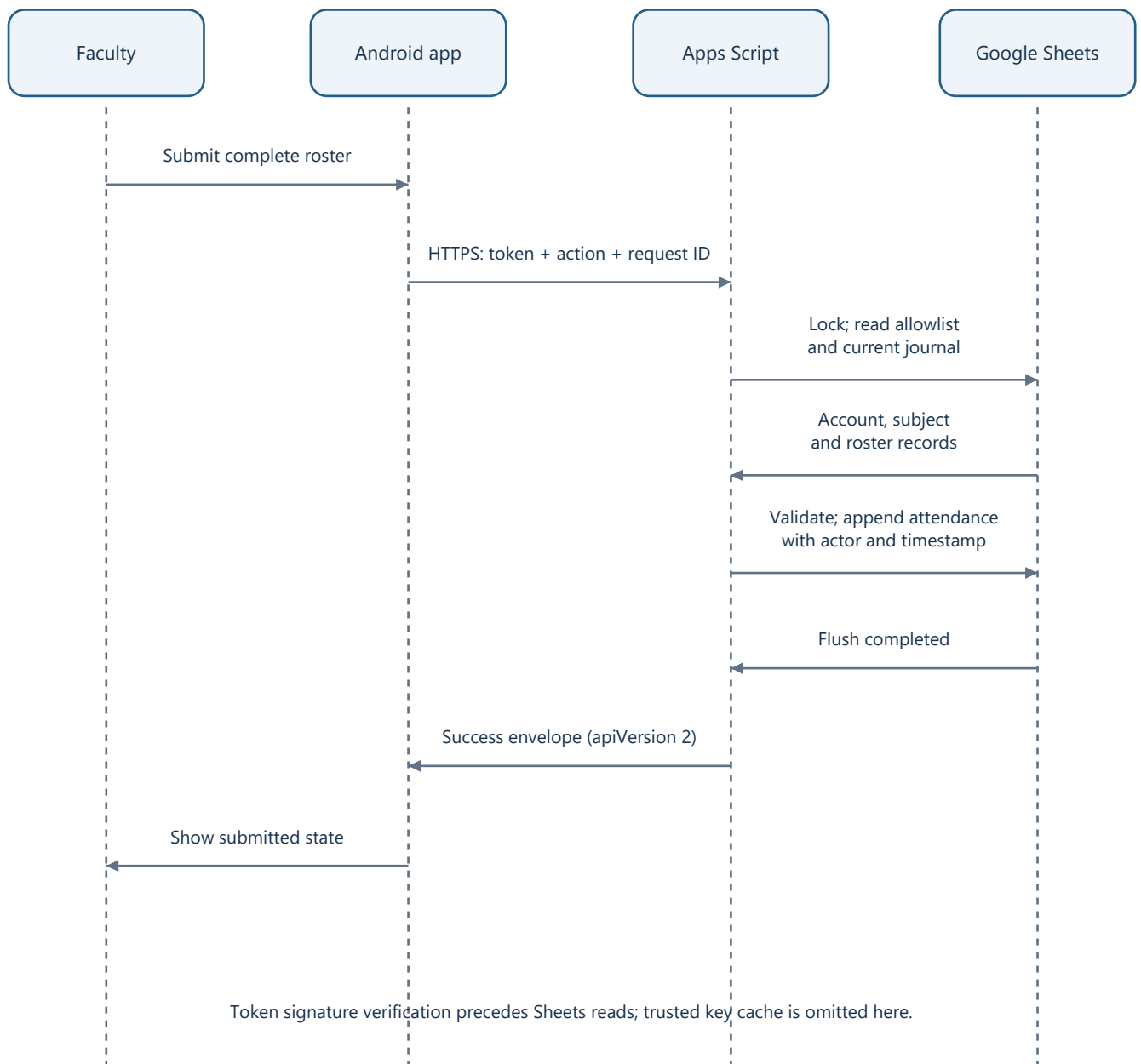


Figure 10. Attendance request sequence diagram

The simplified sequence shows the normal success path. Invalid identity/permissions, changed rosters, duplicate sessions and lock contention return explicit errors without unauthorized writes.

6. Verification and acceptance

6.1 Recorded evidence and remaining limits

Recorded local checks: 27 backend tests and 9 Android JVM tests passed; debug assembly and lint passed with zero errors and 33 remaining warnings. These results are prior verification evidence, not newly executed tests for this document. The example instrumented device test has not been run as a complete acceptance suite.

The demo Apps Script deployment, spreadsheet initialization, OAuth configuration and three role accounts are documented. Real-device Google login and the complete deployed academic workflow remain unverified. A deployed URL and a successful APK build do not establish production readiness.

6.2 Acceptance and traceability matrix

Test / requirement IDs	Acceptance scenario
AT-01 / 01-04	Launch, approved sign-in, expired/forged token rejection, inactive account rejection and logout.
AT-02 / 05-10	Create profiles/subject, reject duplicates/stale edits, prevent unsafe faculty deactivation, enroll distinct active students.
AT-03 / 03, 11-13	Assigned faculty submits full roster; another faculty/student cannot submit it; identical retry yields one session.
AT-04 / 14, 17	Verify 66.7% example, N/A and historical roster eligibility; student sees only own results.
AT-05 / 15-16	Draft accepts pending; publish rejects pending; zero/absent behave correctly; published direct edit rejected.
AT-06 / 18-19	Export Unicode/quoted/formula-like values; open share target; verify loading/error/unsaved-form behavior on phone.
AT-07 / 20, 23-24	Concurrency and lost-response scenarios; approved correction audit and durable draft recovery after implementation.
AT-08 / 21-26, NFRs	Account lifecycle, dated terms/policies, import errors, representative load, backup restore and signed release rehearsal.

7. Roadmap, open decisions and glossary

7.1 Delivery order

Stage	Outcome
Immediate demo acceptance	Install APK; verify all three logins; create/assign subject; enroll student; submit attendance; publish marks; confirm student results and export.
Production domain work	Account administration, academic terms/offerings, dated enrollment, policy settings and approved corrections.
Reliability and reporting	Durable drafts/sync, bulk imports, filtered/printable reports and accessibility improvements.
Mandatory release gates	Capacity validation, separate environments, backup restore, monitoring, release signing, privacy/support ownership and institution acceptance.
Optional extensions	Notification channels, timetable, QR/location aids, assignments, guardian portal, analytics, themes and additional languages.

7.2 Decisions required before production acceptance

Agree attendance counting/shortage and correction policy; grade weightage/retests; term structure; expected institution size; data retention; account approval owner; supported device matrix; backup recovery targets; deployment ownership; and whether any optional feature becomes a requirement.

7.3 Glossary

Term	Meaning
SRS / CSCI	Software Requirements Specification / Computer Software Configuration Item.
ID token / principal	Google-signed identity token / verified actor and permissions used by the backend.
Revision / idempotency	Version used to detect stale edits / repeating the same operation does not create another logical change.
Journal / projection	Append-only record history / latest state reconstructed from that history.
RPO / RTO / p95	Maximum acceptable recovery data loss / recovery time / response time at the 95th percentile.

Document review status: draft for team/institution review. No reviewer approval or production sign-off is implied.